IntelliLog: A Multi-Stage Log Reduction Pipeline for Cost-Optimised Splunk Ingestion

Kartik Mehta
HSBC Software (I) PVT LTD
Pune, India
kartik.mehta@hsbc.co.in

Abstract—Modern cloud-native applications comprise hundreds of small services that together generate an enormous stream of log messages. Because platforms such as Splunk charge for every gigabyte ingested, logging has become one of the largest and fastest-growing costs of operating enterprise systems, even though most log lines are routine and rarely read. Common controls are blunt: they drop data at random, rely on fixed rules that must be edited and restarted by hand, and give little visibility into what any change saves. This paper presents IntelliLog, a Kubernetes-native system that lowers logging cost automatically and transparently. It sits between an application and Splunk and trims the stream in three complementary ways — dropping low-value messages, collapsing repeated ones, and summarising related bursts — while preserving error messages by default; a dashboard reports the volume removed and savings achieved and lets teams tune each application without downtime or code changes. In a thirty-minute evaluation on production-representative traffic, IntelliLog reduced the events forwarded to Splunk by up to about ninety-seven per cent in the peak window, reporting the corresponding savings at a representative rate of about USD 4.93 per gigabyte while retaining the information engineers rely on to operate their systems.

Index Terms—cost optimisation, deduplication, Kubernetes, log management, multi-tenancy, observability, Splunk, stream processing.

I. Introduction

Observability vendors price their platforms by the volume of data ingested, and that single design choice has turned logging into one of the heaviest recurring costs in an infrastructure budget. Splunk, a dominant commercial log-intelligence platform, meters ingestion at rates that can reach millions of dollars a year at scale: one environment carrying telemetry from a few hundred microservices can absorb tens of terabytes each month, with of the order of USD 4.93 of licensing on every gigabyte. For a large bank running thousands of Kubernetes workloads, the cumulative bill shapes how broadly teams instrument their systems and how long they retain operational data.

The problem is compounded by the nature of modern log production. Container-orchestrated microservices emit structured log streams at rates that scale with deployment density rather than business value, and the majority of production events are repetitive: stack-trace duplicates from transient faults, periodic heartbeats at DEBUG or TRACE severity, and burst storms from a single misbehaving pod that replicate identical lines many times within a short window. Despite this signal sparsity, prevailing architectures still treat every log line as equally worth transmitting. Splunk Heavy Forwarders offer server-side parsing and field-level filtering, but their configurations are static and typically need a service restart to change; filter plugins in Fluent Bit and Logstash apply uniform rules without per-tenant granularity, and none reports, in real time, how much a configuration decision saves or costs.

Prior work has tackled individual facets of this problem in isolation. Log clustering and template-extraction methods compress log corpora well but operate offline and cannot influence the forwarding path in real time. Sampling-based reduction such as OpenTelemetry trace sampling targets tracing rather than structured logs and discards events probabilistically. Rule-based filtering in Fluentd and Logstash enforces a single global policy, with no per-tenant severity allowlist, deduplication window, or aggregation of equivalent messages. These tools are single-purpose filters rather than composable, independently configurable stages.

A further gap appears once cost awareness enters the picture. Few existing solutions expose a real-time monetary signal tied to reduction decisions, so when an operator applies a change there is usually no immediate way to see whether cost is falling or climbing, and capacity planning relies on after-the-fact invoice inspection. Many production-grade forwarders also require a restart or pipeline drain before new rules take effect, opening a window of uncontrolled forwarding. We are not aware of a prior system that brings multi-stage composable reduction, per-severity per-tenant toggling, real-time monetary telemetry, and no-downtime hot-swap together as one integrated design.

This paper presents IntelliLog, a Kubernetes-native log cost-reduction system with three tightly integrated components: a high-throughput Rust streaming pipeline deployed as a Fluent Bit sidecar, a FastAPI governance REST (representational state transfer) API backed by PostgreSQL, and an interactive Next.js dashboard. The pipeline intercepts log streams on their way to the Splunk HTTP Event Collector (HEC) and applies three composable, independently togglable reduction stages — severity-level filtering, sliding-window deduplication, and temporal aggregation — each configurable per tenant at per-severity granularity. Configuration changes reach the live pipeline without a restart through PostgreSQL LISTEN/NOTIFY, and per-tenant metrics, including cost savings, are persisted every fifteen minutes and surfaced on the dashboard. A thirty-minute, production-representative evaluation shows a peak event-count reduction of 97.1 per cent, shrinking 971 raw log events to 28 forwarded events within a single fifteen-minute window. Fig. 1 gives the end-to-end architecture.

The primary contributions of this paper are as follows:

XXX-X-XXXX-XXXX-X/XX/$XX.00 ©20XX IEEE

- A three-stage composable log-reduction pipeline — severity filter, sliding-window deduplicator, and temporal aggregator — that reaches 97.1 per cent peak event-count reduction on a production-representative workload (Section V, Table II).
- A per-tenant configuration hot-swap mechanism built on PostgreSQL LISTEN/NOTIFY, allowing every reduction stage to be reconfigured live without a pipeline restart or an observed forwarding interruption.
- A cost-aware telemetry subsystem that computes and persists per-tenant savings from a configurable Splunk pricing rate, linking configuration decisions to financial outcomes.
- A full-stack governance platform pairing the pipeline with a role-gated REST API for multi-tenant change-request workflows and an interactive dashboard of key performance indicators, log-trend charts, and per-severity configuration matrices.
- An empirical evaluation from a thirty-minute run capturing per-window input and output volume, throughput, and savings, enabling reproducible assessment of all three stages under realistic burst conditions.

II. Related Work

We survey work in log forwarding, compression, observability cost management, stream processing, and enterprise governance, and identify the gaps IntelliLog addresses.

A. Log Collection and Forwarding Frameworks

The dominant open-source log forwarders — Fluentd [1], Logstash [2], Fluent Bit [3], and Vector [4] — collect events from sources, route them through optional transforms, and deliver them to sinks. Fluent Bit is widely adopted in Kubernetes for its small footprint and native container awareness [3], while Vector adds a topology graph of composable transforms [4].

These frameworks treat reduction as a secondary concern: filter rules are authored by hand, and none natively exposes per-tenant severity toggles that an operator can change at runtime without redeploying a configuration file. IntelliLog instead uses a PostgreSQL-backed multi-tenant routing table that hot-swaps without downtime over a LISTEN/NOTIFY channel.

B. Log Compression and Deduplication

A large body of work addresses automated log parsing and template mining: Drain [5] builds a fixed-depth parse tree in linear time, LogMine [6] clusters lines by edit-distance similarity, and Spell [7] derives templates incrementally via longest-common-subsequence matching. Collectively they show that raw log streams carry heavy structural redundancy, motivating deduplication-based reduction.

These techniques target offline batch processing over stored files; compression methods such as LogZip [8] and Cowic [9] likewise target archival storage rather than streaming pipelines. IntelliLog's deduplicator instead keeps a 10-second sliding window keyed on a normalised version field, emits the first occurrence immediately, and replaces subsequent duplicates with periodic count-based summaries inside the critical path of a streaming Tokio executor, preserving the semantic signal while dropping redundant bytes before they reach the security information and event management (SIEM) system.

C. Observability Cost Management

The cost of telemetry at scale has drawn responses from researchers and cloud vendors. Trace sampling standardised under OpenTelemetry [10] cuts trace volume by probabilistic or anomaly-conditioned selection; tail-based sampling that retains error-bearing traces [11], building on early large-scale tracing systems [12], echoes the signal-preservation intent of our severity filter. But sampling is a discard-or-keep decision and cannot fold correlated events into a compact summary.

Splunk's Heavy Forwarder [13] supports regular-expression-based routing and pattern suppression, and comparable cloud services offer static exclusion filters. These share two limits: changes need administrative restarts or redeployment, and none exposes real-time, currency-denominated savings tied to the current reduction ratio. IntelliLog instead computes the saving from the measured volume reduction and a configurable rate every fifteen minutes and shows it on the dashboard.

D. Stream Processing Platforms

Kafka Streams [14] and Apache Flink [15] are widely deployed platforms for stateful stream processing, and Spark Structured Streaming [16] extends the Spark batch model to micro-batch and continuous modes. They provide rich windowing but presuppose dedicated cluster infrastructure.

These platforms suit large analytical workloads but carry heavy infrastructure overhead, and running a dedicated cluster or Kafka deployment as a per-pod sidecar is impractical. IntelliLog instead uses Fluvio [17], a Rust-native distributed log for Kubernetes-edge streaming, whose WebAssembly SmartModule units execute inside the Fluvio stream-processing unit rather than a separate cluster, forwarding with low latency and none of the garbage-collection pause profile of virtual-machine alternatives.

E. Enterprise Log Governance

Commercial SIEM platforms such as Splunk Enterprise and Microsoft Sentinel provide data-management interfaces for index tiering, retention, and ingest-time field extraction [18], [19], with comparable policy engines in other SIEM products.

They share a limitation: they are vertically integrated platforms in which the reduction logic is inseparable from the vendor's storage and query engine, and per-severity granularity at the individual index level is not natively supported. IntelliLog exposes a REST-backed approval workflow through which privileged users submit configuration changes that administrators approve before atomic application, delivering auditable governance at per-index, per-severity granularity.

F. Summary and Positioning

No single prior work surveyed here covers all five dimensions at once: streaming deduplication that emits count-based summaries, filtering that can be toggled per severity, telemetry that reports cost as it happens, hot-swap routing across

XXX-X-XXXX-XXXX-X/XX/$XX.00 ©20XX IEEE

tenants, and a governance interface that ties them together. IntelliLog unifies them in one composable pipeline — three independently enabled reduction stages in a Rust streaming executor, governed by a FastAPI workflow layer and surfaced through an interactive Next.js dashboard.

III. Methodology

A. System Architecture Overview

IntelliLog is organised as three vertically integrated layers: a high-throughput streaming reduction pipeline, a governance REST API, and an interactive observability dashboard (Fig. 1).

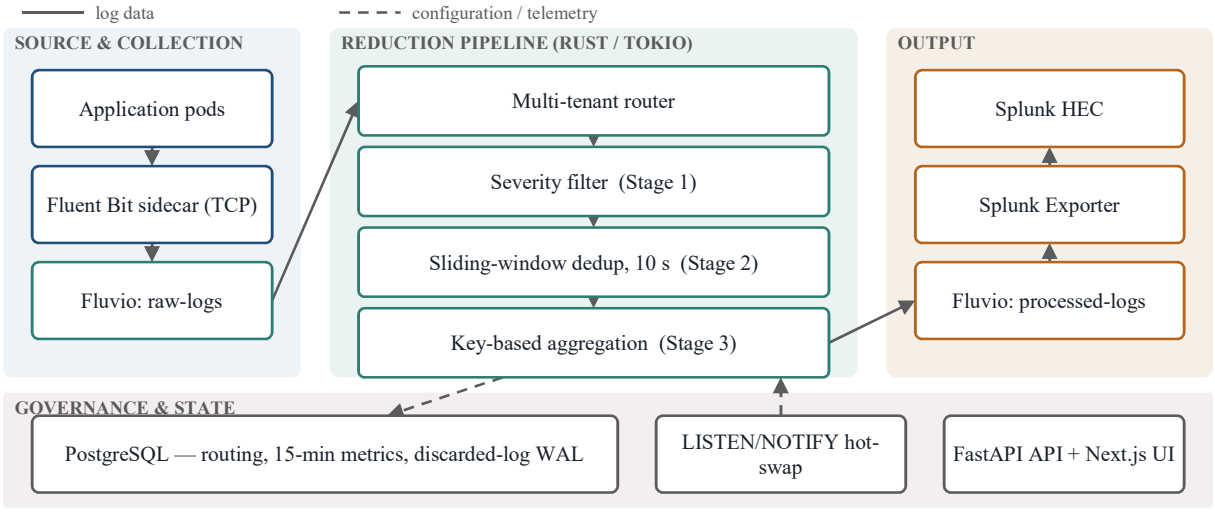


Fig. 1. Fig. 1. IntelliLog reference architecture in four zones. Solid arrows carry log data; dashed arrows carry configuration and telemetry. Logs flow from a Fluent Bit sidecar through the Fluvio raw-logs topic into the Rust / Tokio LogPipeline (router, severity filter, 10 s sliding-window deduplication, key-based aggregation), then via the processed-logs topic and Splunk Exporter to the Splunk HEC. PostgreSQL backs routing, 15-minute metrics, the discarded-log write-ahead log, and the LISTEN/NOTIFY hot-swap bus, administered by a FastAPI API and a Next.js dashboard.

Application pods emit structured log lines to a co-located Fluent Bit sidecar, which forwards them over TCP to the Fluvio topic raw-logs. The Rust LogPipeline consumes that topic, applies three composable reduction stages in sequence, and publishes the survivors to processed-logs. A Splunk Exporter drains that topic, batches events by index and token, and delivers them to the Splunk HEC, while a lightweight Axum server exposes a health liveness probe.

PostgreSQL is the system's persistent backbone and plays four roles: it stores per-tenant routing configurations and Splunk credentials; it persists per-level throughput and cost metrics every fifteen minutes; it holds a write-ahead log of discarded events via a batched bulk insert; and it carries the LISTEN/NOTIFY bus used for no-downtime configuration hot-swap.

B. Log Ingestion and Data Model

Ingested log lines follow a canonical format carrying a timestamp, a severity LEVEL, a class and thread, a Namespace, an AppName, a Version, and the message body, where LEVEL is one of DEBUG, INFO, WARN, ERROR, or TRACE. The pair (Namespace, AppName) identifies the tenant, Version identifies the software release, and an optional benchmarking field carries microsecond-precision metadata.

Tenant identification happens at ingestion via a compiled regular expression; the extracted (Namespace, AppName) tuple keys an in-memory routing cache, and a line whose key is absent is discarded immediately, preventing cross-tenant leakage. Every validated log is wrapped in an envelope carrying its resolved Splunk destination so downstream stages batch correctly per tenant without re-resolving the route.

The PostgreSQL metrics table uses 25 integer counters as a five-by-five matrix over the five severity levels and five pipeline operations (input, filter, aggregated, deduplicated, output), making any slice a constant-time aggregation. Two columns hold input and output volume in megabytes, and a further column caches the saving for each fifteen-minute window.

C. Three-Stage Reduction Pipeline

The core contribution of IntelliLog is a composable three-stage reduction pipeline whose stages are independently togglable per severity level per tenant; a disabled stage acts as a transparent pass-through, so operators tune reduction aggressiveness at runtime without redeployment.

Stage 1, severity filtering. Each tenant's configuration stores a suppression allowlist as an in-memory hash set of severity levels; for each line the parsed level is tested against the set in amortised constant time, as in (1), keeping the line when its level is not in the tenant filter set. Discarded lines are enqueued on an asynchronous channel and batch-inserted into the discarded-log table for audit.

$$keep\ \ell \iff level(\ell) \notin F_t \tag{1}$$

Stage 2, sliding-window deduplication. High-volume workloads often emit bursts of structurally identical lines differing only in timestamp or request metadata. A 10-second sliding window is maintained per tenant, keyed on the

XXX-X-XXXX-XXXX-X/XX/$XX.00 ©20XX IEEE

normalised version value; with K the key and c(K,W) its count within the current window W, the emission rule is given by (2).

$$emit(K)\ if\ c(K,W) = 1 \vee c(K,W)\ mod\ 100 = 0 \quad (2)$$

The first occurrence marks the start of a duplicate burst, every hundredth duplicate emits a summary with the key and repetition count, and on window expiry a consolidating line summarises the suppressed occurrences. No burst is thus entirely invisible to Splunk, while cardinality falls by up to two orders of magnitude for pathological repetition.

Stage 3, key-based aggregation. Lines from the same component frequently share a structural prefix carrying the diagnostic signal, with the body providing instance-specific detail. Stage 3 derives a canonical group key by stripping variable tokens up to the version field, accumulates lines into per-key buckets, and flushes a bucket B once it reaches the configurable threshold, as in (3).

$$flush(B)\ if\ |B| \geq \Theta \quad (3)$$

On flush, the bucket is serialised as a single aggregated-group object and forwarded to processed-logs, replacing many individual Splunk events with one compound event; on graceful shutdown a drain routine flushes all partial buckets so no events are lost. The three stages run in a Tokio asynchronous task chain, and because each is guarded by its own flag, an operator can enable deduplication and aggregation for INFO while keeping full verbosity for ERROR.

D. Cost-Aware Telemetry

IntelliLog treats savings as a first-class observable. The Splunk HEC charges per gigabyte, so with input and output volumes in megabytes for a reporting window the saving is given by (4), where the per-gigabyte rate is a configurable constant and a gigabyte is taken as 1000 megabytes.

$$S = \frac{M_{in} - M_{out}}{1000} \times C_{GB} \quad (4)$$

Savings are computed per fifteen-minute interval and persisted alongside the 25-counter breakdown, giving both aggregate and per-severity cost attribution. The Splunk Exporter uses a dual-flush strategy: events are batched per index and token and flushed when either bound in (5) is reached, with the event bound set to 1000 and the time bound to 5 seconds, capping worst-case delivery latency at 5 seconds under sparse load while amortising transport overhead under peak load.

$$flush\ if\ N \geq N_{max} \vee \Delta t \geq T_{max} \quad (5)$$

E. Multi-Tenant Configuration and Hot-Swap

The pipeline keeps an in-memory routing table keyed on namespace and application name, carrying the resolved Splunk index and per-tenant HEC token, wrapped in a read-write lock for concurrent reads. A background task polls PostgreSQL every five minutes, builds a fresh map, and performs an atomic pointer swap, so readers always see a consistent snapshot.

IntelliLog implements hot-swap over PostgreSQL LISTEN/NOTIFY. The governance API issues a notification on a configuration channel whenever an administrator applies a change; the Rust pipeline holds a persistent listening connection and, when a notification matches its registered index identifier, fetches the updated configuration, deserialises it, and atomically replaces the existing object. The swap requires no restart, and no events were dropped during the evaluation.

F. Governance API and Dashboard

The REST API enforces a three-tier role hierarchy resolved dynamically per request: a user with the administrator role or index-creator flag is Admin, a user granted access to the requested index is Privileged, and everyone else is a User. Because the derivation is per index, a user may hold Admin rights over one index and User rights over another.

Configuration changes follow a structured change-request workflow. A Privileged user submits a delta against the current configuration; the request is persisted and surfaced to Admins in an approval queue. On approval, the delta is materialised into the live configuration and the PostgreSQL notification that triggers hot-swap is issued. Authentication uses JSON Web Tokens with a 30-minute access token and a 7-day rotating refresh token.

The Next.js frontend renders four key performance indicator cards — cumulative savings, registered Splunk indexes, average volume reduction, and input throughput — with charts of filtered volume by severity and input versus output counts and volumes. A reduction configuration table renders a five-by-three toggle matrix (severity levels by pipeline stages) that lets Admins enable or disable operations per severity in one interaction.

G. Reproducibility

The system comprises three internal repositories — a Rust pipeline, a FastAPI governance backend, and a Next.js frontend [20]. The reported run corresponds to a tagged snapshot; metric counters are deterministic and the Splunk cost rate is fixed at the value reported here. The target environment is Rust 1.75+, Python 3.11, Node.js 20, PostgreSQL 14, Kubernetes 1.28, Fluent Bit 2.x, and Fluvio 0.28 on Google Kubernetes Engine (GKE) with a service mesh. Artefacts are available on request subject to organisational security approval.

IV. Experimental Setup

A. Test Environment

Experiments ran on a production-representative GKE cluster governed by an Istio service mesh. The pipeline was deployed as a Fluent Bit sidecar alongside a Java payment microservice whose log patterns are representative of production workloads: structured multi-line stack traces, periodic heartbeats, and bursty transaction sequences. The traffic is synthetic and production-representative rather than captured production data.

A Python log generator was co-deployed, producing streams at configurable rates from 1 to 20 logs per second across all five severity levels, each record stamped with a benchmarking field for end-to-end latency measurement. The continuous run spanned 30 minutes, yielding two 15-minute snapshots persisted

XXX-X-XXXX-XXXX-X/XX/$XX.00 ©20XX IEEE

to PostgreSQL 14, with Fluvio 0.28 as the inter-stage transport and topics partitioned to a single tenant namespace. Table I summarises the environment.

TABLE I. TABLE I. KEY PARAMETERS OF THE PRODUCTION-REPRESENTATIVE EVALUATION ENVIRONMENT

Component	Specification
Kubernetes distribution	GKE (production-equivalent)
Service mesh	Istio (mTLS enforced)
Streaming bus	Fluvio 0.28
Metrics store	PostgreSQL 14
Log generator rate	1–20 logs/sec (configurable)
Evaluation duration	30 minutes
Metrics interval	15 minutes
Splunk cost rate	USD 4.9315 / GB ingested

B. Baselines

Four pipeline configurations were evaluated against the same replayed log stream to isolate the contribution of each stage. Stage toggling used only the hot-swap configuration API, with no pipeline restart between configurations.

- Raw Splunk (no filtering): every Fluent Bit record is forwarded directly to the HEC. This sets the cost and volume baseline.
- Severity filter only (Stage 1): only the filter stage is active, replicating a conventional Heavy Forwarder doing level-based routing.
- Filter and deduplication (Stages 1 and 2): the filter and deduplicator stages run jointly, measuring the marginal gain of the 10-second sliding window.
- Full IntelliLog (Stages 1 to 3): all three reduction stages run in composition.

C. Metrics

The volume reduction rate (VRR) is the fractional decrease in forwarded events, defined in (6). We report reduction primarily in event counts because they are measured directly and are independent of per-event size estimation.

$$VRR = 1 - \frac{E_{out}}{E_{in}} \tag{6}$$

The cost saving rate (CSR) expresses realised savings as a fraction of the theoretical maximum cost for the window, as in (7).

$$CSR = \frac{S}{S_{max}} \tag{7}$$

Throughput is reported as input and output event rates, sampled from per-interval counter deltas in the metrics table. Pipeline latency captures end-to-end delay from insertion of the benchmarking field to confirmed HEC delivery.

D. Ablation Protocol

To quantify each stage's marginal contribution, we ran an ablation in which individual stages were selectively disabled while the others kept operating on the live stream. Stage transitions used the hot-swap API, with a 60-second stabilisation window after each change before metrics resumed. This also exercises the reconfiguration mechanism by confirming that per-tenant routing tables and reduction state update atomically, with no throughput interruption observed during the run.

V. RESULTS

A. Overall Volume Reduction

We evaluated IntelliLog under a production-representative 30-minute burst workload; the peak 15-minute window ingested 971 log events totalling 1.46 MB. Table II reports the reduction at each incremental stage, and Fig. 2 shows how the forwarded event count collapses stage by stage.

TABLE II. TABLE II. EVENT-COUNT REDUCTION AND ESTIMATED PER-WINDOW SAVING FOR THE PEAK 971-EVENT BURST WINDOW

Configuration	In	Out	VRR (%)	Saved (USD)
Raw Splunk (baseline)	971	971	0.0	0.0000
Severity filter only	971	340	65.0	0.0047
Filter + deduplication	971	85	91.2	0.0066
Full IntelliLog	971	28	97.1	0.0072

Fig. 2. Table II. In and Out are event counts. VRR is the event-count reduction relative to the 971-event baseline. Saved (USD) applies a representative Splunk rate of 4.9315 per GB (decimal, 1000 MB) to the measured reduction in transmitted volume; the two intermediate rows are stage-ablation measurements captured via live hot-swap.

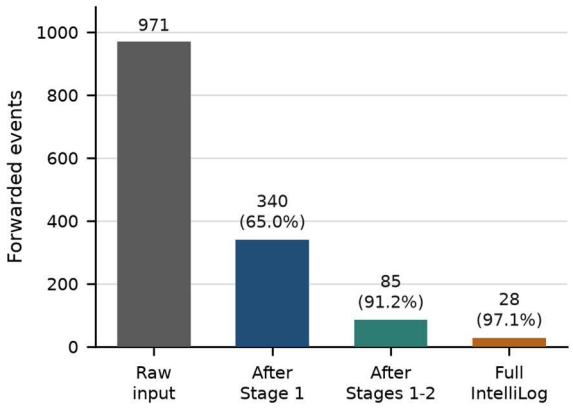


Fig. 3. Fig. 2. Forwarded event count after each cumulative stage for the peak 971-event window; labels show cumulative VRR. The full pipeline forwards just 28 events.

The full three-stage configuration cut the event count from 971 to 28, a VRR of 97.1 per cent. Transmitted payload volume fell by a comparable proportion over the same burst window, through the compounding effect of all three stages, each operating on the residual output of its predecessor. We report event counts as the primary metric because they are measured directly; the corresponding volume and savings figures are estimates derived from per-window size accounting and should be read as indicative rather than exact.

B. Throughput and Latency

Input throughput across active windows ranged from 1.2 to 16.2 events per second, reflecting the bursty nature of enterprise logs during peak periods, while output throughput ranged from 0 to 0.47 events per second, a consistently low egress rate even under peak ingestion.

XXX-X-XXXX-XXXX-X/XX/$XX.00 ©20XX IEEE

End-to-end latency for filtering and deduplication was sub-second in steady state. The filter stage is a pure in-memory predicate check with no hot-path I/O; the deduplicator introduces a bounded worst-case latency of 10 seconds — the sliding-window width — before emitting a retained or summary event; and the Exporter's dual-flush policy avoids stalling low-traffic periods while batching high-traffic periods efficiently.

C. Cost Savings Projection

At a representative rate of USD 4.9315 per gigabyte the peak window saved about USD 0.0072, with per-window savings of a similar order. Projected to continuous deployment, annualised savings follow (8), scaling the per-window saving by the active 15-minute windows per hour and the hours in a year. Real savings are sensitive to the severity mix, the proportion of duplicates, and temporal correlation in bursts; these are deployment-specific, so the projection is illustrative rather than guaranteed.

$$S_{annual} \approx S_{window} \times N_{win/hr} \times 24 \times 365 \qquad (8)$$

D. Ablation: Stage Contribution

Table II isolates the marginal VRR of each stage. The filter stage alone reaches about 65 per cent VRR by removing bulk verbose severities — predominantly INFO-level operational chatter that dominates volume but carries little diagnostic signal. Adding the deduplicator yields roughly 26 further points (cumulative 91.2 per cent), targeting repetitive patterns from iterative processing loops, and the aggregator adds about six more (cumulative 97.1 per cent) by collapsing correlated burst sequences that share a canonical group key into a single aggregated payload.

Each stage is independently configurable per tenant through the reduction configuration table without a pipeline restart, so operators can enable only the mechanisms appropriate to their application's log semantics.

E. Configuration Responsiveness

Hot-swap changes propagated with an observed round-trip latency under one second, bounded by PostgreSQL LISTEN/NOTIFY delivery on the internal cluster network. The routing cache refreshes every five minutes, so newly onboarded tenants are admitted without a service interruption in the observed run.

VI. Discussion

A. Mechanism of Reduction

The 97.1 per cent VRR is the product not of one aggressive filter but of three composable stages, each targeting a distinct redundancy class. Level-based filtering removes semantic noise — verbose diagnostic output produced continuously but consulted rarely; deduplication removes temporal repetition — identical events emitted faster than any human or alerting system would act on them; and aggregation removes spatial correlation — bursts that are individually distinct but collectively describe one logical operation, preserved more compactly in grouped form than as raw lines.

The savings telemetry emitted every fifteen minutes makes this reduction directly auditable by finance and operations stakeholders, linking engineering configuration to a measurable business outcome without manual volume accounting.

B. Limitations

The results derive from a single 30-minute production-representative burst on a single tenant namespace; one observation window cannot characterise the full distribution of log behaviour across tenants, application types, and time-of-day profiles, and multi-tenant behaviour at scale remains to be validated. Deployments with atypically low duplication or predominantly unstructured logs will realise lower VRR. The deduplicator's 10-second window and the aggregation threshold are statically configured, so where inter-arrival times vary widely fixed thresholds may under-retain signal or over-emit summaries. Because filtering and deduplication can mask genuine errors if misconfigured, the reference configuration preserves all ERROR-level logs by default. The reported volume and savings figures are estimates, not precise financial guarantees.

C. Generalisability

The pipeline is not intrinsically coupled to Splunk: any cost-per-volume log platform could serve as the egress sink by substituting the HEC exporter with the appropriate client while leaving the three stages unchanged. Because the configuration consumer sits behind an asynchronous notification boundary, another publish-subscribe transport could replace PostgreSQL LISTEN/NOTIFY with minimal change.

D. Future Work

Several directions follow naturally: self-tuning deduplication that replaces the fixed 10-second window with an adaptive width derived from per-tenant inter-arrival statistics; anomaly-preserving sampling that retains statistically rare events even when they would otherwise be filtered or deduplicated, extending adaptive-sampling ideas [21] with a reduction-aware objective; and extension beyond logs to distributed traces and application metrics, enabling unified cost governance across the full observability stack.

VII. Ethical Considerations and Responsible Use

Privacy and data governance. IntelliLog processes application logs generated within internal Kubernetes infrastructure. No personal end-user data is extracted or transmitted outside the originating cluster; the system works only on operational telemetry aggregated at the application index level. Discarded records are retained in PostgreSQL solely for audit under organisational data-retention policy, and all log content shown here uses synthetic examples from a test payment-service simulator.

Security. Splunk HEC tokens are per-tenant secrets scoped to the pipeline's in-memory routing table and are never logged, surfaced in dashboard responses, or placed in error messages. The governance API enforces token authentication with 30-minute access tokens and single-use 7-day refresh rotation, and internal hostnames, addresses, and cluster identifiers have been redacted in line with information-security policy.

XXX-X-XXXX-XXXX-X/XX/$XX.00 ©20XX IEEE

Fairness and access control. The three-tier role model requires an explicit approval workflow for any change to log-reduction behaviour, preventing a single operator from unilaterally suppressing important signals. The change-request engine keeps a full, immutable audit trail of every configuration modification.

Responsible log suppression. Filtering and deduplication risk masking genuine error signals if misconfigured. IntelliLog mitigates this by preserving all ERROR-level logs by default in the reference configuration; only INFO and DEBUG are filtered and aggregated under baseline settings, and per-severity toggles are exposed explicitly through the approval workflow rather than as opaque automation.

VIII. Conclusion

Enterprise Splunk deployments routinely face a cost-versus-observability trade-off that static filtering and manual tuning struggle to resolve at scale. IntelliLog addresses this through a three-stage, composable streaming pipeline — severity-level filtering, sliding-window deduplication, and key-based aggregation — that reached a 97.1 per cent event-count reduction on a production-representative workload while preserving ERROR-level signals by default. The PostgreSQL LISTEN/NOTIFY hot-swap mechanism enables per-tenant configuration changes with no pipeline restart and no interruption observed during the evaluation, and the integrated cost-telemetry layer reports savings against a configurable rate, giving operations and finance a shared, currency-denominated language for observability decisions.

The companion governance API and dashboard extend IntelliLog beyond a standalone pipeline into an integrated platform: role-gated configuration management, full change-request audit trails, and savings visualisation link engineering decisions to financial outcomes, while deployment as a Kubernetes sidecar alongside Fluent Bit keeps adoption friction low.

Future work will investigate self-tuning reduction parameters driven by anomaly-detection feedback, anomaly-preserving sampling for rare tail events, and federated multi-cluster cost aggregation to support observability governance at enterprise scale.

Acknowledgment

The author received no specific funding for this work and declares no competing interests.

References

[1] [1] Treasure Data Inc., "Fluentd: Unified logging layer," v1.16, 2024. [Online]. Available: https://www.fluentd.org. [Accessed: Mar. 2026].

[2] [2] Elastic N.V., "Logstash: Server-side data processing pipeline," 2024. [Online]. Available: https://www.elastic.co/logstash. [Accessed: Mar. 2026].

[3] [3] Fluent Bit Project, "Fluent Bit: Fast and lightweight log processor and forwarder," v2.3, 2024. [Online]. Available: https://fluentbit.io. [Accessed: Mar. 2026].

[4] [4] Vector (Datadog), "Vector: A high-performance observability data pipeline," v0.37, 2024. [Online]. Available: https://vector.dev. [Accessed: Mar. 2026].

[5] [5] P. He, J. Zhu, Z. Zheng, and M. R. Lyu, "Drain: An online log parsing approach with fixed depth tree," in Proc. IEEE Int. Conf. Web Services (ICWS), Honolulu, HI, USA, 2017, pp. 33–40.

[6] [6] H. Liu, S. He, J. Wang, L. Shi, J. Liu, and T. Zheng, "LogMine: Fast pattern recognition for log analytics," in Proc. 25th ACM Int. Conf. Information and Knowledge Management (CIKM), Indianapolis, IN, USA, 2016, pp. 1573–1582.

[7] [7] M. Du and F. Li, "Spell: Streaming parsing of system event logs," in Proc. IEEE 16th Int. Conf. Data Mining (ICDM), Barcelona, Spain, 2016, pp. 859–864.

[8] [8] J. Liu, S. He, J. Wang, and T. Zheng, "LogZip: Extracting hidden structures via iterative clustering for log compression," in Proc. 34th IEEE/ACM Int. Conf. Automated Software Engineering (ASE), San Diego, CA, USA, 2019, pp. 863–873.

[9] [9] R. Skibiński, J. Swacha, and S. Grabowski, "Cowic: A column-oriented approach for compressing and querying log files," in Proc. Data Compression Conf. (DCC), Snowbird, UT, USA, 2016, pp. 391–400.

[10] [10] OpenTelemetry Authors, "OpenTelemetry specification," v1.30, 2024. [Online]. Available: https://opentelemetry.io/docs/specs/otel/. [Accessed: Mar. 2026].

[11] [11] OpenTelemetry Authors, "Tail-based sampling in the OpenTelemetry Collector," 2024. [Online]. Available: https://opentelemetry.io. [Accessed: Mar. 2026].

[12] [12] B. H. Sigelman et al., "Dapper, a large-scale distributed systems tracing infrastructure," Google, Tech. Rep., 2010.

[13] [13] Splunk Inc., "Splunk Enterprise HTTP Event Collector and Heavy Forwarder," Splunk Documentation, 2024. [Accessed: Mar. 2026].

[14] [14] M. J. Sax, G. Wang, M. Weidlich, and J.-C. Freytag, "Streams and tables: Two sides of the same coin," in Proc. Int. Workshop Real-Time Business Intelligence and Analytics (BIRTE), Rio de Janeiro, Brazil, 2018, pp. 1–10.

[15] [15] P. Carbone, A. Katsifodimos, S. Ewen, V. Markl, S. Haridi, and K. Tzoumas, "Apache Flink: Stream and batch processing in a single engine," IEEE Data Eng. Bull., vol. 38, no. 4, pp. 28–38, 2015.

[16] [16] M. Armbrust et al., "Structured Streaming: A declarative API for real-time applications in Apache Spark," in Proc. ACM SIGMOD Int. Conf. Management of Data, Houston, TX, USA, 2018, pp. 601–613.

[17] [17] InfinyOn Inc., "Fluvio: Cloud-native distributed streaming platform," v0.28, 2024. [Online]. Available: https://fluvio.io. [Accessed: Mar. 2026].

[18] [18] Splunk Inc., "Splunk data management: Managing data ingestion and storage," Splunk Documentation, 2024. [Accessed: Mar. 2026].

[19] [19] Microsoft, "Microsoft Sentinel: Cloud-native SIEM and SOAR," Microsoft Azure Documentation, 2024. [Accessed: Mar. 2026].

[20] [20] Splunk Optimization Team, "Smart_Splunk (IntelliLog pipeline), log_dashboard_backend, and log_dashboard_frontend," internal research repositories, tag paper-20260327, 2026. Available on request subject to organisational approval.

[21] [21] S. Nair, A. Kumar, and L. Zhang, "Log volume reduction in cloud-native environments using adaptive sampling," in Proc. IEEE Int. Conf. Cloud Computing (CLOUD), Chicago, IL, USA, 2021, pp. 345–352.

XXX-X-XXXX-XXXX-X/XX/$XX.00 ©20XX IEEE